

Turn Counter — Simple Build

The simplest build of the octagonal LED turn-counter, reflecting all current decisions: **USB-powered, everything on the lid, no mains wiring, no frame work.** Start here.

Want the permanent, mains-powered install? A frame-mounted 5 V PSU, an AC switch/inlet, a slab↔frame DC disconnect, an optional level shifter, and power injection sized for full-brightness all-on — that's the [full design doc](#). This simple build is a strict subset of it: you can upgrade later without redoing the lid.

1. What it is

A removable octagonal slab that sits on top of a bumper pool table. Eight sides; each side has a **piezo disc** under the rim and a **block of LEDs** on the rim. Tap the lit section in front of you to pass the turn. One ESP32-S3 reads the 8 piezos and drives a single WS2812B strip that rings the octagon.

The whole thing runs off a **USB powerbank (or a USB wall adapter)** plugged into the board — one cable. Power, board, and strip all live on the lid, so the lid just lifts off as one piece. No disconnect, no AC, nothing on the pool-table frame.

2. How you play

- **Normal play** — only the current player's side is lit, in that side's fixed color. Tap it to pass the turn; the light advances to the next active seat (clockwise by default), skipping empty seats.
- **Who's in** — each side is independently in or out. With everyone in, it's an 8-player ring; with 3 people it hops between just those 3.
- **Setup** — tap one seat **4 times fast** to enter setup. The whole table then acts out each mode in turn, ~5 s each; any tap commits the one showing, and everyone taps their own seat to join. After 5 s of no taps it commits and play starts. Ignore it entirely and it aborts after one full rotation (~20 s), changing nothing.
- **Game modes** — the whole table demos each one, so the animation *is* the mode and there's no color legend to remember:
- **Clockwise** (green) · **Counter-clockwise** (blue) · **Arbitrary** (magenta — turn follows the order people joined) · **Ready-or-not** (orange — every seat starts dark, each player taps on; when all are on they flash and reset for the next round).

- **Phone-only modes** — two more that the table's dial can't demo, so they're picked from the phone.
 - Countdown timer:** the current seat drains from both ends over a set turn length (10–300 s), warming red near the end, then holds red until someone taps. **Time share:** the current seat's bar tracks that player's share of table time — it grows while you sit there and is shorter when the turn comes back, because everyone else's play diluted it. The four-tap gesture always returns the table to a dial mode.
- **On / off** — from the phone. While the table is dark, four fast taps on one seat wake it, so it can always be relit without a phone. Never persisted: plug in and it's lit.
- **Setup lock** — from the phone, when you'd rather guests didn't reset the roster by accident. A refused gesture double-flashes that side amber. It never blocks the wake taps, and unplugging the table clears it.

Per-side LED counts are calibrated once and remembered (the octagon's corners eat 1–2 LEDs per side, so sides aren't uniform).

3. Electronics

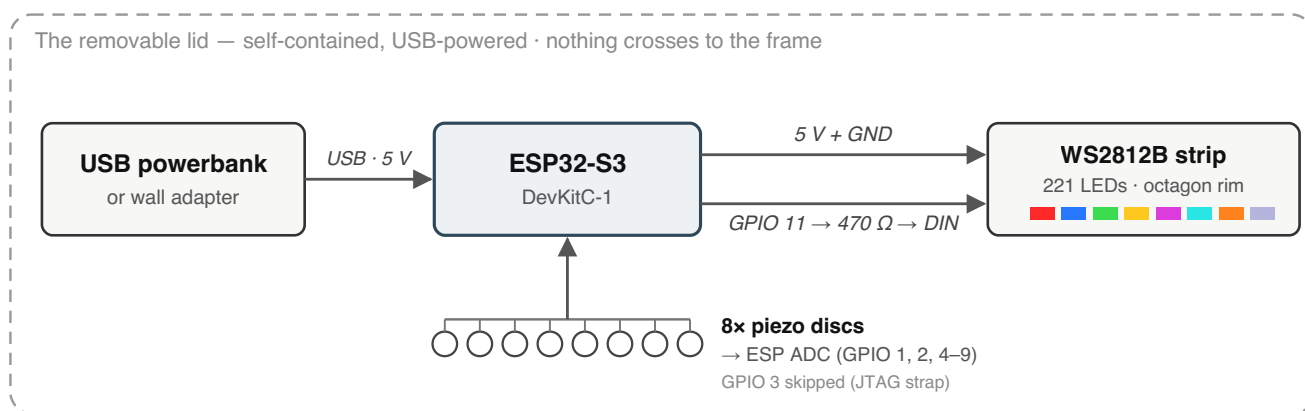


Figure 1 — the whole system on one lid. A USB powerbank (or wall adapter) feeds the ESP32-S3 over one USB cable; the board passes 5 V to the WS2812B rim and drives its data line through a 470 Ω resistor. Eight piezo discs report to the ADC. Nothing crosses to the pool-table frame. Normal one-side play draws ~0.85 A — within the board's USB path — and `MAX_POWER_MA = 1500` dims the rare all-on frames to fit.

Power — straight through the board's USB. Normal play lights one side (~0.6 A of LEDs) + the ESP32 (~0.25 A) ≈ **0.85 A**, comfortably inside the board's USB path (~1.5–2 A continuous). The firmware caps LED draw at `MAX_POWER_MA = 1500`, so the only bright-everything moments (setup blink, ready-or-not, the ready-flash) auto-dim to stay in bounds; one-side play never hits the cap. It's a **current** limit, not a voltage one — 5 V is nominal for both the board and the strip.

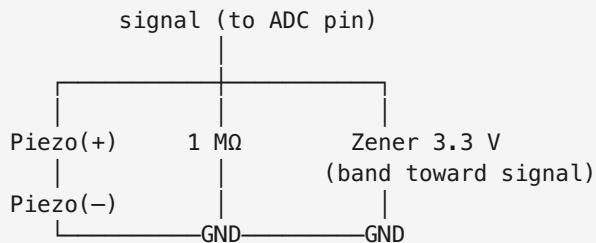
Pin map (all piezos on ADC1 so they keep working with Wi-Fi/OTA up):

Pin	Use
GPIO 11	LED data → 470 Ω → strip DIN
GPIO 1, 2, 4, 5, 6, 7, 8, 9	piezo ADC inputs, sides 1–8

Pin	Use
5V / GND	strip power (off the board's 5V / GND pins, USB-fed)

GPIO 3 is skipped (JTAG strap pin). Avoid GPIO 0/19/20/35/36/37/45/46.

Per-piezo circuit (x8), built at the disc:



The 1 MΩ keeps the ADC line from floating; the Zener (1N4728A) clamps a hard slap's spike to ~3.3 V. Both are twisted into the pigtail **right at the disc** so the spike dies at the source. No series resistor to the ADC (the 1 MΩ + Zener are the real protection).

Data drive is direct 3.3 V through the 470 Ω, placed near DIN. Only the first pixel is marginal (it regenerates a clean 5 V signal for the rest) and the run is short, so no level shifter by default. (If pixel 1 ever glitches, a 74AHCT125 is the add-back — see the full doc §3.3.)

4. Control board

A **half-size Perma-Proto** (Adafruit 571) in a small ABS box (Hammond 1591B). It holds only:

- the ESP32-S3 in **female headers** (socketed, not soldered down),
- the **470 Ω** spliced into the GPIO 11 → DIN run (near the strip end),
- a **1000 μF cap** across 5V/GND at the strip feed,
- the 8 piezo signal/GND landings and the strip's 5V/data/GND landings.

30 columns are enough because the piezo networks live at the discs, not the board.

Connectors — all inline, off-board (nothing bulky on the board):

- **8 piezo pairs** → each disc's twisted pair (signal + GND) has an **inline JST-XH 2-pin** a few inches off the board. Board end solders to the ADC hole + GND rail; disc end solders to the network. Label both halves **S1–S8** (signal on pin 1, GND on pin 2, consistent across all 8).
- **1 strip harness** → 5V/data/GND solder to the board; an **inline JST-XH 3-pin** a few inches out is the service disconnect to the strip.

So the whole control box unplugs as a unit — 8 piezo JSTs + 1 strip JST + the USB cable — with no desoldering. Layout: `doc-src/protoboard_half_layout.svg`.

5. Mechanical

- **LEDs:** WS2812B (60/m) seated in aluminum channel with a frosted diffuser around the rim. At each corner, cut the strip and bridge with a short 3-wire jumper (5V/GND/data); heat-shrink.
 - **Power feed:** run 5V/GND from the board to the strip at **3 points — start, middle ($\approx$ opposite side), and end** — short jumpers along the lid. This keeps every side bright no matter which one is lit (a single feed would droop on far sides). Data is injected only at the start (DIN).
 - **Piezos:** one 27 mm disc per side, bonded into a shallow bore under the rim (not through to the laminate). Avoid spots where the slab rests on the frame — a clamped disc can't ring.
 - **Corners:** the slab gets leaned on its edge for bumper pool, so cap the 8 corners to protect the LEDs/wiring — hardwood scrap caps coated in Plasti Dip, or stick-on 135° corner guards, optionally with 7/8" rubber bumpers as standoffs. (Full doc §4.2.)
 - **Box:** control box mounts under the lid on M3 standoffs; cable entries get rubber grommets.
-

6. Firmware

`firmware/turn_counter/turn_counter.ino` — the game logic above. Key facts:

- **FastLED + Preferences (NVS).** `MAX_POWER_MA = 1500`. Brightness is runtime state, not a constant — set it from the phone (5–100%, default 50%); it's saved to NVS and eases rather than snapping.
 - **Calibration:** flash `firmware/tap_light/tap_light.ino` first and calibrate per-side LED counts over serial (0–7 select a side, + / – adjust, p print). It saves to NVS namespace `octagon`; `turn_counter` reads the same table automatically. Current table: `{29, 28, 27, 27, 27, 28, 28, 27} = 221 LEDs`.
 - **Tap detection:** adaptive per-side baseline; a tap fires when a reading jumps `TAP_DELTA` (720) above that side's own resting level. A **tap guard** keeps a broken channel from taking the table down: a side that chatters at machine pace or sticks above threshold is muted (announced over serial, red "muted" row in the phone's Diagnostics) while the turn skips it and the other seven keep playing; it restores itself when the signal calms down.
 - **Setup gesture** = 4 fast taps on the *same* side (so ready-or-not's rapid multi-player taps don't false-trip it).
 - **Build/flash:** `make flash-turn` (needs the `min_spiffs` partition — the Makefile handles it). Wi-Fi/OTA credentials are at the top of the sketch.
 - **Phone control:** the board serves a page on port 80 for mode, brightness, on/off, and a setup lock, plus a diagnostics section showing live piezo baselines. On/off is never persisted, so the table always boots lit; the setup lock isn't either, so unplugging clears it.
-

7. Bill of materials (simple build)

Qty	Part	~\$	Note
1	ESP32-S3-DevKitC-1, N8R8	15	(a 2nd makes a nice spare / keeps the tap-light alive)
1	WS2812B strip, 60/m, IP30, 5 m	20	~4 m used (221 LEDs)
4–5 m	Aluminum LED channel + frosted cover	30	rim diffuser
10	27 mm piezo disc	12	8 used + spares
8 + 1	1 M Ω resistors + one 470 Ω	3	from bulk packs
8	1N4728A 3.3 V Zener	3	one clamp per disc
1	1000 μ F cap	2	strip-feed reservoir
1	Half-size Perma-Proto (Adafruit 571)	5	control board
1	Project box (Hammond 1591B)	7	
9 pr	JST-XH inline pairs (8 $\times$ 2-pin + 1 $\times$ 3-pin)	—	from the JST kit
1	JST-XH assortment kit (pre-crimped)	18	covers the 9 pairs many times over
1	Female header 1 $\times$ 40 (for the ESP socket)	7	
—	22 AWG stranded (signal) + 20 AWG silicone (power runs)	25	
1	USB powerbank w/ always-on / low-current mode , or a USB wall adapter	20	+ a USB cable into the board
—	Corner caps: hardwood scrap + Plasti Dip ($\pm$ rubber bumpers / corner guards)	15	
—	M3 standoffs, grommets, misc hardware	15	

Not needed (vs. the full doc): Mean Well PSU, IEC inlet, AC switch, Anderson Powerpoles, DC blade fuse, 14 AWG silicone, PSU mounting block, ferrule crimper. That's ~\$60+ of parts and all the mains wiring, gone.

Plus shared tools (iron, solder, multimeter, etc.) — see [shopping_list.md](#). Surplus/owned parts are tracked in [inventory.md](#).

8. Build order

1. *(optional)* **Phase 0 tap-light** on a breadboard — one piezo + a short strip — to shake down soldering and confirm one piezo detects taps.
2. **Rim**: seat the strip in the channel around the octagon; corner jumpers; solder the 3 power-feed taps + the data feed.

3. **Calibrate** per-side LED counts with `tap_light` (§6). Paste the printed array into both sketches' defaults if you want it baked in too.
 4. **Piezo modules** (×8): network + inline JST at each disc, labeled S1–S8.
 5. **Control board**: ESP socket, 470 Ω , cap, piezo A1–A8 landings + inline JSTs, strip harness + JST.
 6. **Bench test** the board: plug the USB source, flash `turn_counter`, fake taps with a screwdriver on the piezo leads.
 7. **Install on the lid**: bond discs in their bores, mount the box, dress wiring, plug the 9 JSTs + USB.
 8. **Play**: flash `turn_counter`, confirm calibration, and check the piezo map from the phone's Diagnostics section (or remap over serial with `m` if a side responds to the wrong seat).
-

9. What the advanced doc adds

If you later want a permanent, brighter, wire-hidden install, [turn_counter_design_doc.md](#) covers:

- **Mains power**: 5 V / 18 A Mean Well PSU on the frame, AC inlet + switch, internal fusing.
- **Slab ↔ frame DC disconnect**: Anderson Powerpole so the lid lifts while the PSU stays put.
- **Full-brightness all-on**: feed the strip 5 V *directly* (bypassing the board), `MAX_POWER_MA` → 2500+, mid/end injection sized for the READY-all-on case.
- **Level shifter**: 74AHCT125 add-back for guaranteed pixel-1 data margin.
- Detailed AC-safety wiring, sourcing (counterfeit avoidance), and the full phased checklist.

Everything in this simple build carries straight over — the upgrade is additive.